REFERENZ

Standardmakros

Makros aus c82ass.makros nach Themen: Aufrufparameter, Wirkung und Mehrfachaufruf

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	C8205_Standardmakros_c82ass.md	0.01-020
Stand	10.11.2026	
Umfang	11 Seiten, A4 hoch	
Erzeugt mit	erzeuge_standardmakros_pdf.py	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

C 8205 – Standardmakros

Quelle: C8205_Standardmakros_c82ass.md · im Wortlaut der Quelle

Inhaltsverzeichnis

0. Gemeinsame Regeln . . . 3

1. Übersicht . . . 3

2. Organisatorisch: Ausgabesteuerung . . . 4

3. Organisatorisch: Geräteanwahl . . . 4

4. Organisatorisch: Trommelübertragung . . . 4

5. Wiederholung, Schieben, Adresseinheit . . . 5

6. Feldzugriff, Index, Zeiger . . . 5

7. Textunterprogramme kat4, kat5, kat6 . . . 6

7.1 kat4 (4 Zeichen je Wort, erstes Zeichen rechts, z0 am letzten Wort) . . . 6

7.2 kat5 (neues Format nach Doku 3.3.15: genau **ein** Wort, 1 bis 5 Zeichen, links beginnend in z3 bis z8, Füllung 00 rechts, kein z0) . . . 6

7.3 kat6 (5 Zeichen je Wort zu 6 Bit im Eingabecode, erstes Zeichen rechts, Stoppbit über dem letzten Zeichen, z0 am letzten Wort; Doku 3.3.18) . . . 7

8. Gleitkomma (GK), Format dwg . . . 8

9. Festkomma (FK), Format dwf . . . 8

10. Spezifikationszeile ($SPEC) der Standardmakros . . . 9

C 8205 – Standardmakros

Quelle: C8205_Standardmakros_c82ass.md · im Wortlaut der Quelle

Version: 0.01-020

Stand: 10.11.2026 · Quelle: `Assembler/Sourcecode/c82ass.makros` (1978 Zeilen, mit `KAT5SORT` und `KAT5SORT2`, jedes Makro mit `$SPEC`-Zeile; Assembler c82ass 1.03-009); Stand der kat5-Abschnitte: Lieferung 030, `KAT5SORT` ergänzt in Lieferung 007, `KAT5SORT2` ergänzt in Lieferung 011 Art: Übersicht, abgeleitet aus den #-Kommentaren der Makrodatei. Die Standardmakros sind nur hier und in diesen Kommentaren beschrieben, nicht in der Assemblerdokumentation; deren Abschnitte 4.25 und 4.26 erklären nur den Makromechanismus. Bei Abweichung gilt der #-Kommentar des Makros in `c82ass.makros`. Alle Zahlen sind oktal, wie in der Quelle.

0. Gemeinsame Regeln

- **Arbeitsspur 7600 bis 7677** (Festlegung 025): Dort darf nichts stehen, was über das Ende einer Befehlsfolge hinaus erhalten bleiben muss. Makros mit Spuroperand (SCHIEBE_*, MACHE_ADRESSEINHEIT2) überschreiben Zellen der Spur 7640.
- **Register der UP-Makros:** R1 = 7773 (abs 07773), R2 = 7776, R3 = 7774. Der Aufrufer trägt die Werte vor dem Aufruf ein (z. B. `211 radr`, `240 07773`). R3 zeigt auf die KB/GB-Zelle des Geräts (`da kbgb`; 0 = KB, 1 = GB, anfangs KB; Festlegung 030).
- **Aufruf eines UP-Makros:** Das Makro erzeugt den Rumpf, der Ablauf läuft nicht hinein. Aufgerufen wird mit `025 <marke>`, das Rückkehrwort steht auf der Zelle `ML0001%` am Ende des UP (kein eigener Rücksprung, Festlegung 010).
- **Generische Marken** im Makro haben die Form `ML0001%`; jeder Aufruf bekommt eigene.
- **Vierstellige Parameter** bekommen eine Vornull (`ANWAHL 4600` erzeugt `030 04600`).
- **Mehrfachbenutzung / Einfachspeicherung:** Die Spalte „Mehrfachaufruf“ nennt je Makro, ob mehrere Aufrufe nur **einmal** Code erzeugen (Einfachspeicherung, über `$GLOBAL`-Merker, c82ass ab 1.02-001) oder bei **jedem** Aufruf neuen Code.
- **Einfachspeicherung (Ja):** KAT4/5/6 AUSGABE und EINGABE, GK_MATH, GK_SIN, FK_MATH.
- **Jeder Aufruf erzeugt Code (Nein):** alle übrigen (WRZL, KB, GB, ANWAHL, TANWAHL, TROUT, TRIN, WDHL, SCHIEBE_*, MACHE_ADRESSEINHEIT*, ARRAY_*, ERHOEHE, ZEIGER_NAECHST); sie sind Einzelbefehle oder kurze Befehlsfolgen und stehen bei jedem Aufruf im Ablauf. Diese 20 Makros bekommen die Spezifikationszeile `$SPEC TYP=A ACTION=None` (Abschnitt 10).
- **Version der Makrodatei:** Sie hat keine eigene Versionsnummer; maßgeblich ist der Assemblerstand (c82ass 1.03-009).

1. Übersicht

Thema	Makros
Organisatorisch: Ausgabesteuerung	WRZL, KB, GB
Organisatorisch: Geräteanwahl	ANWAHL, TANWAHL
Organisatorisch: Trommelübertragung	TROUT, TRIN
Wiederholung, Schieben, Adresseinheit	WDHL, SCHIEBE_LI, SCHIEBE_RE, MACHE_ADRESSEINHEIT, MACHE_ADRESSEINHEIT2
Feldzugriff, Index, Zeiger	ARRAY_LOAD, ARRAY_STORE, ARRAY_ADD, ARRAY_SUB, ARRAY_VGL, ARRAY_SWAP, ERHOEHE, ZEIGER_NAECHST
Text kat4	KAT4AUSGABE, KAT4EINGABE
Text kat5	KAT5AUSGABE, KAT5EINGABE, KAT5SORT, KAT5SORT2
Text kat6	KAT6AUSGABE, KAT6EINGABE
Gleitkomma (GK)	GK_MATH, GK_SIN
Festkomma (FK)	FK_MATH

2. Organisatorisch: Ausgabesteuerung

Makro	Parameter	Kurzbeschreibung	AC / zerstört	Mehrfachaufruf
WRZL	keine	Zeilenschaltung ausgeben (`dz $wrzl`), danach Kanal 0 anwählen (`ANWAHL 4700`). 2 Zellen.	AC unverändert; nichts zerstört	Nein (jeder Aufruf erzeugt Code)
KB	keine	Umschaltung in Kleinbuchstaben ausgeben (`dz $inkb`). 1 Zelle.	AC unverändert; nichts zerstört	Nein
GB	keine	Umschaltung in Großbuchstaben ausgeben (`dz $ingb`). 1 Zelle.	AC unverändert; nichts zerstört	Nein

3. Organisatorisch: Geräteanwahl

Makro	Parameter	Kurzbeschreibung	AC / zerstört	Mehrfachaufruf
ANWAHL	`$1` = Kanal: `4000` `4100` `4200` `4300` `4400` `4500` `4600` `4700`, oder `K0` bis `K7`, oder `LSL1` `LSL2` `LKL1` `LKS1` `LSS1` `SW1I` `SW1O` `SW2O` `KONI`	Anwahl Einzelzeichen-Ein-/Ausgabe; erzeugt `030 0xxxx` (Ausgabe-Halt ohne Löschen).	AC unverändert; nichts zerstört	Nein
TANWAHL	`$1` = Zusatztrommel: `5100` `5200` `5300` `5400` oder `ZTR1` bis `ZTR4`	Anwahl der Zusatztrommel; erzeugt `030 05k00`.	AC unverändert; nichts zerstört	Nein

Zuordnung bei ANWAHL:

Code	Namen
4000	K7
4100	K6
4200	K5
4300	K4, LKS1
4400	K3, LKL1, SW2O
4500	K2, LSL2, SW1O
4600	K1, LSL1, LSS1
4700	K0, SW1I, KONI

4. Organisatorisch: Trommelübertragung

Beide nur mit `--C8205Z` wirksam; die Zusatztrommel vorher mit `TANWAHL` anwählen.

Makro	Parameter	Kurzbeschreibung	AC / zerstört	Mehrfachaufruf
TROUT	`$1` = Haupttrommeladresse, `$2` = Zusatztrommeladresse (je Zahl 0 bis 07777 oder abs-Symbol, Spuranfang, Vielfache von 040)	Ausgabe Haupttrommel `$1` nach Zusatztrommel `$2`; ein Befehlswort `zzz0 bbbb 601`.	AC gelöscht; Spur `$2` der Zusatztrommel überschrieben	Nein
TRIN	`$1` = Zusatztrommeladresse, `$2` = Haupttrommeladresse (Wertebereich wie TROUT)	Eingabe Zusatztrommel `$1` auf Haupttrommel `$2`; ein Befehlswort `zzz0 bbbb 661`.	AC gelöscht; Spur ab `$2` der Haupttrommel überschrieben	Nein

5. Wiederholung, Schieben, Adresseinheit

Makro	Parameter	Kurzbeschreibung	AC / zerstört	Mehrfachaufruf
WDHL	benannt: loeschen = JA/NE; resultat = LI/RE/ZL/OHNE; operation = KON/ADD/SUB/BADD; anzahl = 1 bis 32; spur = Adresse ($ADRESSE)	Operation anzahl-fach mit den Zellen der Spur ab dem berechneten Sektor, danach Resultatsbehandlung; bei JA wird der AC vorher gelöscht. Erzeugt !$wdhl $spur (Doku 4.26).	AC = Ergebnis; Zellen werden nur gelesen	Nein
SCHIEBE_LI	$1 = Stellenzahl n, 1 bis 32	AC n Stellen nach links (nicht zyklisch, Bits gehen verloren); Wiederholungsbefehl 644 auf Spur 7640 (Festlegung 026). 1 Zelle.	AC verschoben; Spur 7640 vom Operandsektor bis Spurende	Nein
SCHIEBE_RE	$1 = Stellenzahl n, 1 bis 32	AC n Stellen nach rechts (logisch, links mit 0 aufgefüllt); 642 auf Spur 7640. 1 Zelle.	AC verschoben; Spur 7640 wie oben	Nein
MACHE_ADRESSEINHEIT	$1 = Adresse des Werts, $2 = Zieladresse	Wert laden, durch 9 Linksverschiebungen in Adresseinheiten umrechnen (mal 01000), speichern nach $2. Ablauf: 215, 8 mal 004 *+1, 240, WDHL JA LI KON 9 004000 (11 Zellen, gezählt).	AC = 0; nur $2 zerstört; Arbeitsspur nicht benutzt	Nein
MACHE_ADRESSEINHEIT2	$1 = Adresse des Werts, $2 = Zieladresse, $3 = Verschiebezahl 1 bis 32 (für Adresseinheiten 9)	Wert laden, über SCHIEBE_LI $3 verschieben, speichern. 3 Zellen. Keine eigene Nullspur.	AC = verschobener Wert (auch in $2); $2 und Spur 7640	Nein

6. Feldzugriff, Index, Zeiger

Bei allen ARRAY_*: $1 ist die **Zelle, die die Feldadresse enthält** (da feld), nicht das Feld selbst; $2 (und $3 bei SWAP) sind **Indexzellen in Adresseinheiten** (Wert mal 01000, siehe MACHE_ADRESSEINHEIT). Die Makros **bilden nur den Befehl** und speichern ihn auf die Zielzelle; diese muss im Ablauf unmittelbar dort stehen, wo der Befehl laufen soll (db 1, Festlegung 023). Je 4 Zellen (211, 210, 210, 240).

Makro	Parameter	Gebildeter Befehl (Adresse = $1 + $2)	AC / zerstört	Mehrfachaufruf
ARRAY_LOAD	$1 Feldadresszelle, $2 Indexzelle, $3 Zielzelle für den Befehl	211 laden	AC = fertiger Befehl; nur $3 zerstört	Nein
ARRAY_STORE	wie ARRAY_LOAD	240 speichern	wie oben	Nein
ARRAY_ADD	wie ARRAY_LOAD	210 addieren	wie oben	Nein
ARRAY_SUB	wie ARRAY_LOAD	220 subtrahieren	wie oben	Nein
ARRAY_VGL	wie ARRAY_LOAD	330 vergleichen	wie oben	Nein
ARRAY_SWAP	$1 Feldadresszelle, $2 und $3 Indexzellen, $4 Hilfszelle	Vertauscht $1[$2] und $1[$3]; läuft ohne Sprung durch (20 Zellen).	AC = ursprüngliches Element $1[$2]; zerstört $4, vier Befehlszellen ML0001% bis ML0004%, beide Feldelemente (vertauscht)	Nein (20 Zellen je Aufruf)

Makro	Parameter	Gebildeter Befehl (Adresse = $1 + $2)	AC / zerstört	Mehrfachaufruf
ERHOEHE	$1 = Indexzelle (Adresseinheiten)	Zelle um eine Adresseinheit (01000) erhöhen: 211, 210 #01000, 240. 3 Zellen.	AC = neuer Wert; nur $1 zerstört	Nein
ZEIGER_NAECHST	$1 = Zeigerzelle (da text)	Gleiche Befehlsfolge wie ERHOEHE; der Name zeigt nur, dass $1 ein Zeiger ist.	AC = neuer Zeiger; nur $1 zerstört	Nein

7. Textunterprogramme kat4, kat5, kat6

Alle sechs sind **UP-Makros ohne Makroparameter**. Aufruf: name: KATnXXX (Marke auf der ersten Zelle), dann Register füllen, Kanal anwählen (ANWAHL), 025 name.

Mehrfachaufruf (gilt für alle sechs, Einfachspeicherung): Der Rumpf entsteht nur beim ersten Aufruf im Lauf (globale Variable <makro>_da, $GLOBAL, auch mit ;#MAKROLOKAL=1); jeder weitere Aufruf erzeugt nichts. Die Marke steht nur beim ersten Aufruf auf dem UP. Alle 025 müssen auf die Marke des ersten Aufrufs zeigen; spätere Aufrufe schreibt man ohne Marke (eine Marke dort bekäme die Adresse der nächsten Zelle, Doku 4.1). Braucht c82ass ab 1.02-001.

Zustand KB/GB: Das UP zieht die Zelle in R3 nach (Um. GB setzt sie auf 1, Um. KB auf 0) und liest den Anfangszustand nicht. Die Ausgabe-UP geben die Umschaltzeichen selbst nicht von sich aus aus; die Eingabe-UP speichern sie wie jedes Zeichen (kat5/kat6: Um. KB/GB werden mitgespeichert und zählen mit).

Spezifikationszeile ($SPEC, Assemblerdokumentation 4.32): Alle acht Makros dieses Abschnitts enthalten Textunterprogramme und bekommen als zweite Zeile hinter MACRO die Zeile $SPEC TYP=M CLASS=KAT4 ACTION=None (KAT4AUSGABE, KAT4EINGABE), $SPEC TYP=M CLASS=KAT5 ACTION=None (KAT5AUSGABE, KAT5EINGABE, KAT5SORT, KAT5SORT2) bzw. $SPEC TYP=M CLASS=KAT6 ACTION=None (KAT6AUSGABE, KAT6EINGABE). Stand 09.11.2026: in c82ass.makros eingetragen, seit c82ass 1.03-002 ausgewertet (Näheres: Abschnitt 10).

7.1 kat4 (4 Zeichen je Wort, erstes Zeichen rechts, z0 am letzten Wort)

Makro	Register (Aufrufparameter)	Kurzbeschreibung	AC / zerstört
KAT4AUSGABE	R1 = Adresse des Textes (da text); R3 = Adresse der KB/GB-Zelle	Gibt einen kat4-Text über den angewählten Kanal aus. 47 Zellen. Um. GB = kat4 057, Um. KB = kat4 117.	AC = letztes Textwort (mit z0); zerstört UP-Zellen und Spur 7640; R1 bleibt
KAT4EINGABE	R1 = Adresse des Bereichs (da text); R2 = höchste Zeichenzahl (mindestens 1); R3 = KB/GB-Zelle	Liest Zeichen von K0 (SW / T-SW) und legt sie als kat4-Text ab. Ende mit WRZL (nicht gespeichert) oder sobald R2 Zeichen angenommen sind; Speicher (R2+3)/4 Worte; das letzte Wort bekommt z0. Zeichentabelle 64 Zellen auf 0100-Grenze (UP etwa 0146 Zellen bis zur Tabelle, gemessen).	AC = Zeichenrest (negativ); zerstört UP-Zellen, Spur 7640, Tabelle samt Behandlern; R1, R2, R3 bleiben

7.2 kat5 (neues Format nach Doku 3.3.15: genau ein Wort, 1 bis 5 Zeichen, links beginnend in z3 bis z8, Füllung 00 rechts, kein z0)

Codes: 01 Leerzeichen, 02 ,, 03 -, 04 ., 05 +/=, 06 :, 07 bis 020 Ziffern 0 bis 9, 021 bis 052 a bis z, 053 WRZL, 054 Um. KB, 055 Um. GB. Jeder Aufruf behandelt genau ein Wort; für längere Texte ruft der Aufrufer mehrfach auf und stellt R1 jedes Mal um 01000 (eine Adresseinheit) weiter.

Makro	Register (Aufrufparameter)	Kurzbeschreibung	AC / zerstört
KAT5AUSGABE	R1 = Adresse des Wortes (da wort); R3 = KB/GB-Zelle	Gibt die Zeichen eines kat5-Wortes aus. Ende, sobald der Rest nach dem letzten Zeichen 0 ist (Füllung 00 oder nach 5 Zeichen); Wort 0 gibt nichts aus. Ausgabe über Tabelle ML0013% (45 Zellen, Index = kat5-Code, liegt hinter dem UP). Um. KB/GB (054, 055) werden ausgegeben und ziehen R3 nach. Unzulässige Codes (00 vor einem Zeichen, 056 bis 077, gesetztes z0 bis z2) werden nicht geprüft.	AC = 0; zerstört UP-Zellen (ML0004%, ML0005%, ML0009%, ML0012%, ML0001%) und Spur 7640; R1, R3 bleiben

Makro	Register (Aufrufparameter)	Kurzbeschreibung	AC / zerstört
KAT5EINGABE	R1 = Adresse des Wortes (da wort); R3 = KB/GB-Zelle; **R2 wird nicht benutzt**	Liest höchstens 5 Zeichen von K0 und legt sie als **ein** Wort auf R1. Ende nach 5 Zeichen oder mit WRZL (nicht gespeichert, Zeichen linksbündig, rechts 00; WRZL als erste Taste ergibt Wort 0). Bei vollem Wort bleibt eine folgende Taste für den nächsten Aufruf stehen. Gespeichert werden a bis z, 0 bis 9, Leertaste, . , - +, : (kein Ende mehr) sowie Um. KB/GB; andere Tasten werden überlesen (nicht gespeichert, nicht gezählt). Zeichentabelle ML0026% (64 Zellen) auf 0100-Grenze, Einträge sind die kat5-Codes selbst.	AC = gespeichertes Wort (nie negativ); zerstört UP-Zellen, Spur 7640; R1, R3 bleiben
KAT5SORT	Parameterzellen im UP: Marke+1 = Feldanfang, Marke+2 = Anzahl der Wörter (gewöhnliche Zahl); Aufruf 025 <Marke>	Sortiert ein Feld von kat5-Wörtern aufsteigend auf denselben Zellen (Wortwert = Sortierreihenfolge). n <= 1 kehrt sofort zurück. 66 Zellen (oktal) in einem Stück, ohne Tabelle. **Der Rumpf entsteht nur beim ersten Aufruf** ($GLOBAL kat5sort_da): weitere Aufrufe erzeugen nichts; Marke und Parameter nur über die Marke des ersten Aufrufs ansprechen (sort+1, sort+2). Beispiel: Assembler/Claude/kat5_sort1, Mehrfachaufruf: Assembler/Claude/kat5sort_2x.	AC ohne Bedeutung; zerstört UP-Zellen (ML0001% bis ML0006%, ML0014%, ML0015%) und das Feld (wird sortiert); R1, R3 und die Spur bleiben unberührt
KAT5SORT2	Parameterzellen im UP: Marke+1 = Feldanfang, Marke+2 = Anzahl der **Paare** (gewöhnliche Zahl, nicht der Zellen); Aufruf 025 <Marke>	Sortiert ein Feld aus Paaren (kat5-Wort, Wert) aufsteigend nach dem kat5-Wort auf denselben Zellen und zieht den Wert mit; der Wert wird nie verglichen. Ein Paar sind zwei benachbarte Zellen: kat5-Wort auf a, Wert auf a + 01000, das nächste Paar beginnt bei a + 02000. n <= 1 kehrt sofort zurück. 105 Zellen (oktal) in einem Stück, ohne Tabelle. **Der Rumpf entsteht nur beim ersten Aufruf** ($GLOBAL kat5sort2_da): weitere Aufrufe erzeugen nichts; Marke und Parameter nur über die Marke des ersten Aufrufs ansprechen (sort+1, sort+2). KAT5SORT und KAT5SORT2 haben getrennte Schalter und dürfen im selben Programm stehen. Beispiel: Assembler/Claude/kat5_sort2_makro (dasselbe UP als eigener Rumpf ohne Makro: Assembler/Claude/kat5_sort2).	AC ohne Bedeutung; zerstört UP-Zellen (ML0001% bis ML0007%, ML0013% bis ML0016%) und das Feld (wird sortiert); R1, R3 und die Spur bleiben unberührt

Stand (Lieferung 030): KAT5AUSGABE und KAT5EINGABE laufen im neuen Format; KAT5SORT (Lieferung 007) sortiert ein Feld solcher Wörter, KAT5SORT2 (Lieferung 011) ein Feld aus Paaren (kat5-Wort, Wert) und zieht den Wert mit. Die zehn kat5-Programme unter Assembler/Claude (kat5, kat5makro, kat5ausgabe, _alle, _voll, kat5eingabe, _alle, _um, _voll, _wrzl) sind darauf umgestellt und enden mit „Test wahr“; auch kat_einmal und alle kat4- und kat6-Programme sind wahr. Größe gemessen am Test: KAT5AUSGABE 34 Zellen plus Tabelle 45, KAT5EINGABE 56 Zellen (bis einschließlich der Arbeitszelle ML0030%) plus Tabelle 64 auf der 0100-Grenze. Die Doku (4.25, 3.3.15) ist seit 1.03-002 angepasst.

7.3 kat6 (5 Zeichen je Wort zu 6 Bit im Eingabecode, erstes Zeichen rechts, Stoppbit über dem letzten Zeichen, z0 am letzten Wort; Doku 3.3.18)

Makro	Register (Aufrufparameter)	Kurzbeschreibung	AC / zerstört
KAT6AUSGABE	R1 = Adresse des Textes (da text); R3 = KB/GB-Zelle	Gibt einen kat6-Text aus, ohne Tabelle: Zeichen = (c UND 017) + 2 mal (c UND 060), Ausgabeadresse = Zeichen mal 8. Wort fertig, wenn nur noch das Stoppbit übrig ist; Text endet mit dem Wort mit z0 (Endwort ohne Zeichen gibt nichts aus). Um. GB = kat6 037, Um. KB = kat6 057 (werden ausgegeben). 50 Zellen.	AC = letztes Textwort (mit z0, negativ); zerstört UP-Zellen, Spur 7640; R1, R3 bleiben
KAT6EINGABE	R1 = Adresse des Bereichs (da text); R2 = höchste Zeichenzahl (mindestens 1); R3 = KB/GB-Zelle	Liest Zeichen von K0 und legt sie als kat6-Text ab. Ende (1) mit : (wird als letztes Zeichen gespeichert), (2) mit WRZL (nicht gespeichert) oder (3) nach R2 Zeichen; das letzte Wort bekommt z0, Speicher höchstens (R2+4)/5 Worte. Gespeichert werden a bis z, 0 bis 9, Leertaste, . , - +, : / %, Akzent sowie Um. KB/GB. 96 Zellen plus Tabelle ML0026% (64 Zellen, nur gelesen).	AC = Zeichenrest (negativ); zerstört UP-Zellen, Spur 7640; R1, R2, R3 bleiben

8. Gleitkomma (GK), Format dwg

Format dwg: z0 Vorzeichen, z1 bis z6 Exponent e+32, z7 bis z32 Mantisse (2^25 ≤ M < 2^26), Wert = ±M/2^26 · 2^e, Null = Wort 0.

Makro	Parameter	Kurzbeschreibung	Aufruf / Ergebnis	Mehrfachaufruf
GK_MATH	$1 = add | sub | mul | div | alle (ohne Marke)	UP der Grundrechenarten. Ein Aufruf erzeugt nur den Einsprung des genannten UP (Marke gk_add, gk_sub, gk_mul, gk_div) und, was noch fehlt: Kern der Addition (add, sub), Kern von Multiplikation und Division (mul, div), beim allerersten Aufruf den gemeinsamen Teil. Wer nur gk_add braucht, bekommt weder Multiplikation noch Division.	R1 (7773) und R2 (7776) laden, dann 025 gk_add | gk_sub | gk_mul | gk_div; Ergebnis im AC (dwg): R1+R2, R1−R2, R1·R2, R1/R2. R1, R2, R3 bleiben. Unterste Ebene (ruft kein UP auf). Division durch 0 (auch 0/0) und Exponentenüberlauf: künstlicher Überlauf (AC := 2^32−1, +1, Halt); Unterlauf ergibt 0; Rundung Mitte weg von 0. Befehle je Aufruf: add 12 bis 853, sub 13 bis 858, mul 14 bis 72, div 14 bis 554.	**Ja, je Teil:** jeder Teil des Codes wird nur einmal erzeugt (Merker gkm_add, gkm_sub, gkm_mul, gkm_div, gkm_ad, gkm_md, gkm_pk); darf beliebig oft und mit jedem Parameter aufgerufen werden. Mit zwei Aufrufen liegt der Code an zwei Stellen im Speicher.
GK_SIN	keine (ohne Marke)	Sinus und Kosinus (gk_sin, gk_cos, zweiter Einsprung im selben Code, gk_sin = gk_cos + 1). Der erste Aufruf ruft GK_MATH add, sub, mul selbst auf (Teile, die schon da sind, entstehen nicht noch einmal). Argumentreduktion nach Cody und Waite, Horner-Reihe.	x (dwg, Bogenmaß) nach gks_x (Zelle gk_sin+1) legen, dann 025 gk_sin bzw. 025 gk_cos; Ergebnis im AC (dwg), gks_x bleibt. Bereich |x| < 4096·π/2 (rund 6433), sonst künstlicher Überlauf. Fehler höchstens etwa 2 (sin) bzw. 1,4 (cos) Einheiten der letzten Mantissenstelle. Übergabe über gks_x, nicht über R1 (Festlegung 031). Zerstört auch R1, R2, Spur 7640 und die Zellen von GK_MATH; R3 unberührt. Umfang: gk_sin und gk_cos 0234 Zellen, mit GK_MATH add, sub, mul zusammen 0636.	**Ja:** Code nur einmal (globale Variable gk_sin_da); Marken gk_sin und gks_x stehen nur beim ersten Aufruf, alle Aufrufer benutzen diese Marken.

Namensraum: Innere Marken beginnen mit gkm_ (GK_MATH) bzw. gks_ (GK_SIN); im Programm dürfen keine eigenen Marken gks_*, gk_sin, gk_cos stehen. Prüfprogramme: Projekt_Gleitkomma/gk_math_op, gk_trigonomie_sin, Claude/gk_sin_makro, Claude/gk_cos_makro.

Spezifikationszeile ($SPEC, Assemblerdokumentation 4.32): Alle Standardmakros dieses Abschnitts, die Gleitkomma-Unterprogramme enthalten (GK_MATH und GK_SIN), bekommen als zweite Zeile hinter MACRO die Zeile $SPEC TYP=M CLASS=GK ACTION=None. Stand 09.11.2026: in c82ass.makros eingetragen, seit c82ass 1.03-002 ausgewertet (Näheres und Übersicht aller Makros: Abschnitt 10).

9. Festkomma (FK), Format dwf

Format dwf: Wort = x · 2^32, Zweierkomplement 33 Bit, Komma zwischen z0 und z1, Bereich −1 bis 1 − 2^−32.

Makro	Parameter	Kurzbeschreibung	Aufruf / Ergebnis	Mehrfachaufruf
FK_MATH	$1 = add | sub | mul | div | alle (ohne Marke)	UP der Grundrechenarten. Ein Aufruf erzeugt den Einsprung des genannten UP (Marke fk_add, fk_sub, fk_mul, fk_div), beim ersten Bedarf von mul oder div den gemeinsamen Kern, beim allerersten Aufruf die Rückkehrzelle fkm_rueck. Wer nur fk_add braucht, bekommt keinen Kern. Add und Sub sind je ein Befehl (210, 220).	R1 (7773) und R2 (7776) laden, dann 025 fk_add | fk_sub | fk_mul | fk_div; Ergebnis im AC (dwf). R1, R2, R3 bleiben. Unterste Ebene. Überlauf bei Add/Sub sowie (−1)·(−1), −1/−1 erzeugt die Maschine selbst; Division durch 0 und Quotient mit Betrag ≥ 1: künstlicher Überlauf; a/b mit a = −b ergibt −1, mit a = b Überlauf. Rundung Mitte weg von 0. mul zerstört Spur 7640.	**Ja, je Teil:** jeder Teil nur einmal (Merker fkm_add, fkm_sub, fkm_mul, fkm_div, fkm_md, fkm_pk).

Namensraum: Innere Marken beginnen mit fkm_. Prüfung: Projekt_Festkomma, fk_math_op.

Spezifikationszeile (`$SPEC`, Assemblerdokumentation 4.32): `FK_MATH` (das einzige Standardmakro mit Festkomma-Unterprogrammen) bekommt als zweite Zeile hinter `MACRO` die Zeile `$SPEC TYP=M CLASS=FK ACTION=None`. Stand 09.11.2026: in `c82ass.makros` eingetragen, seit c82ass 1.03-002 ausgewertet (Näheres: Abschnitt 10).

10. Spezifikationszeile (`$SPEC`) der Standardmakros

Jedes Makro kann als zweite Zeile hinter `MACRO` genau eine Spezifikationszeile `$SPEC [TYP=…] [CLASS=…] [ACTION=…]` tragen (Assemblerdokumentation 4.32). Ohne die Zeile gelten die Vorgaben `TYP=M` und `ACTION=None`; für `CLASS` ist keine Vorgabe festgelegt. Die Tabelle weist die Spezifikation jedes Standardmakros aus.

Stand 09.11.2026: Die Zeilen der Spalte „Zeile“ stehen in `c82ass.makros` (alle 31 Makros als zweite Zeile hinter `MACRO`); c82ass wertet sie seit 1.03-002 aus (Prüfung der Schreibweise, Warnung bei `TYP=M` ohne `$GLOBAL` und ohne `$BLOCK`). `$BLOCK` und `$ENDBLOCK` kennt c82ass seit 1.03-003; die Standardmakros benutzen es nicht. Den vierten `TYP`-Wert `E` (Eingebettet, ab c82ass 1.03-004; Assemblerdokumentation 4.32) benutzt **kein** Standardmakro: Er ist für Hilfskonstrukte gedacht, die nur den Quelltext eines Projekts gliedern und nicht mehrfach verwendet werden (Beispiel: die Makros des Projekts ZT-Loader, siehe `C8205_ZT_Makros_Uebersicht.md`, Abschnitt 8). Die Tabelle unten enthält deshalb keine Zeile mit `TYP=E`. Regeln aus 4.32: Ein Makro mit `$SPEC` und `TYP=M` erzeugt seinen Rumpf in einem Lauf nur einmal und setzt dafür einen globalen Merker (`$GLOBAL`); ein Makro mit `TYP=A` darf nie eine Mehrfachauflösung verhindern. Die elf Makros mit Einmalerzeugung (Klassen KAT4, KAT5, KAT6, GK und FK; Abschnitte 7 bis 9) haben `TYP=M` und erfüllen die Regel schon heute: Die Spalte „Merker“ nennt den Merker, der in `c82ass.makros` vorhanden ist (geprüft am Quelltext 09.11.2026). Die übrigen 20 Makros (Abschnitte 2 bis 6) erzeugen bei jedem Aufruf neuen Code (Spalte „Mehrfachaufruf“: Nein), haben keinen Merker und kein `$GLOBAL` (am Quelltext 09.11.2026 geprüft; das `$IF` in `ANWAHL` und `TANWAHL` wählt nur die Schreibweise des Operanden) und bekommen deshalb `$SPEC TYP=A ACTION=None`, ohne `CLASS` (für `CLASS` gibt es keine Vorgabe, und `$BLOCK` brauchen diese Makros nicht). `TYP=A` verhindert nie eine Mehrfachauflösung, der Assembler prüft bei `TYP=A` nichts (4.32); es ändert sich also nichts am Verhalten dieser Makros, die Zeile hält nur ihre Art fest. Damit hat jedes der 31 Standardmakros eine Spezifikationszeile; die Vorgabe `TYP=M` ohne Zeile kommt bei Standardmakros nicht mehr vor.

Umordnung durch den Optimierer (ab c82ass 1.03-007, Assemblerdokumentation 5.6): Der Optimierer darf die Rümpfe der Standardmakros mit `TYP=M` hinter das Programmende verlegen und dicht packen, wenn dadurch die Endadresse kleiner wird; `;#MAKROUMORDNEN=NO` und `--no-opt` schalten das ab. Nach Assemblerläufen betrifft das KAT4AUSGABE, KAT5SORT, KAT5SORT2, KAT6AUSGABE, GK_MATH, GK_SIN und FK_MATH (Gruppe 4). KAT4EINGABE, KAT5EINGABE und KAT6EINGABE (Zeichentabelle mit `equ100 *` hinter der ersten Anweisung) und KAT5AUSGABE (wegen `dto`) bleiben an ihrer Stelle. Ein Programm darf sich daher nicht auf die Lage eines dieser Rümpfe im Quelltext verlassen (feste Adressen, Hineinlaufen, Herauslaufen); die Marke am Aufruf wandert mit dem Rumpf.

Die Klasse `GK` bezeichnet Makros, die Gleitkomma-Unterprogramme enthalten, die Klasse `FK` Makros mit Festkomma-Unterprogrammen (`FK_MATH`), die Klassen `KAT4`, `KAT5` und `KAT6` Makros mit kat4-, kat5- bzw. kat6-Unterprogrammen (7.1 bis 7.3). Die Klasse steht in Großbuchstaben, wie die Definition 4.32 es verlangt.

Makro	Thema	Zeile in der Makrodatei	TYP	CLASS	ACTION	Merker (Einmalerzeugung)
`WRZL`	Ausgabesteuerung	ja: `$SPEC TYP=A ACTION=None`	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
`KB`	Ausgabesteuerung	ja: `$SPEC TYP=A ACTION=None`	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
`GB`	Ausgabesteuerung	ja: `$SPEC TYP=A ACTION=None`	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
`ANWAHL`	Geräteanwahl	ja: `$SPEC TYP=A ACTION=None`	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
`TANWAHL`	Geräteanwahl	ja: `$SPEC TYP=A ACTION=None`	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)

C 8205 – Standardmakros

Quelle: C8205_Standardmakros_c82ass.md · im Wortlaut der Quelle

Makro	Thema	Zeile in der Makrodatei	TYP	CLASS	ACTION	Merker (Einmalerzeugung)
TROUT	Trommelübertragung	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
TRIN	Trommelübertragung	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
WDHL	Wiederholung, Schieben, Adresseinheit	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
SCHIEBE_LI	Wiederholung, Schieben, Adresseinheit	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
SCHIEBE_RE	Wiederholung, Schieben, Adresseinheit	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
MACHE_ADRESSEINHEIT	Wiederholung, Schieben, Adresseinheit	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
MACHE_ADRESSEINHEIT2	Wiederholung, Schieben, Adresseinheit	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
ARRAY_LOAD	Feldzugriff, Index, Zeiger	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
ARRAY_STORE	Feldzugriff, Index, Zeiger	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
ARRAY_ADD	Feldzugriff, Index, Zeiger	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
ARRAY_SUB	Feldzugriff, Index, Zeiger	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
ARRAY_VGL	Feldzugriff, Index, Zeiger	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
ARRAY_SWAP	Feldzugriff, Index, Zeiger	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
ERHOEHE	Feldzugriff, Index, Zeiger	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
ZEIGER_NAECHST	Feldzugriff, Index, Zeiger	ja: $SPEC TYP=A ACTION=None	A	– (keine Klasse)	None	– (kein Merker; jeder Aufruf erzeugt Code)
KAT4AUSGABE	Text kat4	ja: $SPEC TYP=M CLASS=KAT4 ACTION=None	M	KAT4	None	kat4ausgabe_da
KAT4EINGABE	Text kat4	ja: $SPEC TYP=M CLASS=KAT4 ACTION=None	M	KAT4	None	kat4eingabe_da
KAT5AUSGABE	Text kat5	ja: $SPEC TYP=M CLASS=KAT5 ACTION=None	M	KAT5	None	kat5ausgabe_da
KAT5EINGABE	Text kat5	ja: $SPEC TYP=M CLASS=KAT5 ACTION=None	M	KAT5	None	kat5eingabe_da

C 8205 – Standardmakros

Quelle: C8205_Standardmakros_c82ass.md · im Wortlaut der Quelle

Makro	Thema	Zeile in der Makrodatei	TYP	CLASS	ACTION	Merker (Einmalerzeugung)
KAT5SORT	Text kat5	ja: `$SPEC TYP=M CLASS=KAT5 ACTION=None`	M	KAT5	None	`kat5sort_da`
KAT5SORT2	Text kat5	ja: `$SPEC TYP=M CLASS=KAT5 ACTION=None`	M	KAT5	None	`kat5sort2_da`
KAT6AUSGABE	Text kat6	ja: `$SPEC TYP=M CLASS=KAT6 ACTION=None`	M	KAT6	None	`kat6ausgabe_da`
KAT6EINGABE	Text kat6	ja: `$SPEC TYP=M CLASS=KAT6 ACTION=None`	M	KAT6	None	`kat6eingabe_da`
GK_MATH	Gleitkomma (GK)	ja: `$SPEC TYP=M CLASS=GK ACTION=None`	M	GK	None	`gkm_add`, `gkm_sub`, `gkm_mul`, `gkm_div`, `gkm_ad`, `gkm_md`, `gkm_pk` (je Teil)
GK_SIN	Gleitkomma (GK)	ja: `$SPEC TYP=M CLASS=GK ACTION=None`	M	GK	None	`gk_sin_da`
FK_MATH	Festkomma (FK)	ja: `$SPEC TYP=M CLASS=FK ACTION=None`	M	FK	None	`fkm_add`, `fkm_sub`, `fkm_mul`, `fkm_div`, `fkm_md`, `fkm_pk` (je Teil)